Full-Text Search in PostgreSQL

Il full-text search di PostgreSQL trasforma il testo in token normalizzati, li indicizza e permette di cercarli con operatori booleani, frasi, prefix match e ranking di rilevanza. I due tipi centrali sono tsvector, che rappresenta il documento indicizzabile, e tsquery, che rappresenta la condizione di ricerca.

Pipeline di ricerca

```
Testo originale
      |
      v
Parser: identifica token, parole, numeri, URL, ecc.
      |
      v
Dizionario: stemming, rimozione stopword, normalizzazione
      |
      v
tsvector: lexeme + posizioni + pesi
      |
      v
Indice GIN
      |
      v
tsquery: condizione di ricerca
      |
      v
Risultati ordinati per rilevanza (ts_rank)
```

Per esempio, "running" e "runs" possono essere ridotti allo stesso lexeme "run"; parole molto comuni come "the" possono essere scartate come stopword.

Tipi principali

Tipo	Ruolo	Esempio
tsvector	Rappresentazione ottimizzata del testo per la ricerca	'un gatto veloce'::tsvector
tsquery	Query di ricerca con operatori booleani e di frase	to_tsquery('gatto & veloce')
regconfig	Configurazione linguistica	'italian'::regconfig
regdictionary	Dizionario usato per normalizzare un token	english_stem

Un tsvector contiene i lexemi normalizzati, le loro posizioni nel documento e, quando presenti, le etichette di peso A, B, C e D. Un tsquery contiene i lexemi da cercare, combinabili con & (AND), | (OR), ! (NOT) e <-> (FOLLOWED BY).

1. Creare il vettore

```
SELECT to_tsvector('italian', 'La ricerca full-text in PostgreSQL e'' molto utile');
```

La funzione to_tsvector converte il testo in un tsvector usando il parser e i dizionari della configurazione linguistica indicata.

Funzioni principali

Funzione	Uso
to_tsvector([config,] text)	Converte testo in tsvector usando parser e dizionari
strip(tsvector)	Rimuove posizioni e pesi, conservando solo i lexemi
setweight(tsvector, "char")	Assegna un peso a tutti i lexemi del vettore
length(tsvector)	Restituisce il numero di lexemi distinti
tsvector_update_trigger()	Trigger che popola automaticamente una colonna tsvector
to_tsvector(regconfig, text)	Versione con configurazione linguistica esplicita

```
-- Vettore con pesi per titolo e corpo
SELECT
  setweight(to_tsvector('italian', titolo), 'A') ||
  setweight(to_tsvector('italian', corpo), 'B') AS ricerca
FROM articoli;
```

I pesi sono utili per far contare di più il titolo rispetto al corpo: A ha priorità maggiore, seguito da B, C e D.

2. Creare la query

PostgreSQL fornisce quattro funzioni per convertire l'input utente in tsquery. La scelta dipende da quanto controllo vuoi dare all'utente e dalla complessità della sintassi che vuoi supportare.

Funzione	Input tipico	Comportamento
to_tsquery	gatto & veloce	Interpreta operatori espliciti; input non valido genera errore
plainto_tsquery	gatto veloce	Ignora punteggiatura e crea AND tra i termini
phraseto_tsquery	gatto veloce	Crea una ricerca di frase con <->
websearch_to_tsquery	"gatto veloce" OR cane	Sintassi stile motore di ricerca, più sicura per input utente

```
-- AND implicito: gatto AND veloce
SELECT plainto_tsquery('italian', 'gatto veloce');
-- 'gat' & 'veloc'

-- Frase esatta: gatto seguito da veloce
SELECT phraseto_tsquery('italian', 'gatto veloce');
-- 'gat' <-> 'veloc'

-- Sintassi web: frase, OR e esclusione
SELECT websearch_to_tsquery('italian', '"gatto veloce" OR cane -persiano');
```

websearch_to_tsquery è in genere la scelta migliore per una casella di ricerca pubblica: supporta frasi tra virgolette, OR, - per escludere termini e non richiede all'utente di conoscere la sintassi tsquery.

Operatori tsquery

Operatore	Significato	Esempio
&	AND: tutti i termini devono essere presenti	'gatto' & 'veloce'
|	OR: almeno uno dei termini deve essere presente	'gatto' | 'cane'
!	NOT: esclude i documenti con quel lexeme	'!'persiano'
<->	FOLLOWED BY: i termini devono essere adiacenti nell'ordine	'gatto' <-> 'veloce'
<N>	FOLLOWED BY con distanza massima	'gatto' <2> 'veloce'
:*	Prefix match	'postgr:*'

```
SELECT *
FROM articoli
WHERE ricerca @@ to_tsquery('italian', 'gatto & !persiano');

SELECT *
FROM articoli
WHERE ricerca @@ to_tsquery('italian', 'postgr:*');
```

Gli operatori <-> e cercano termini nello stesso ordine e con una distanza limitata; sono utili per frasi e per ricerche più precise.

3. Matching e ranking

L’operatore @@ confronta un tsvector con un tsquery e restituisce true in caso di corrispondenza. Le funzioni ts_rank e ts_rank_cd assegnano invece un punteggio di rilevanza ai risultati.

Funzione	Restituisce	Uso
@@	boolean	Verifica se un tsvector corrisponde a un tsquery
ts_match_vq(tsvector, tsquery)	boolean	Equivalente funzionale di @@
ts_match_qv(tsquery, tsvector)	boolean	Ordine invertito degli argomenti
ts_rewrite(tsquery, tsquery, tsquery)	tsquery	Sostituisce una parte della query
ts_rewrite(tsquery, text)	tsquery	Applica regole di riscrittura da una tabella

```
SELECT *
FROM articoli
WHERE ricerca @@ websearch_to_tsquery('italian', 'gatto veloce');
```

Ranking

Funzione	Uso
ts_rank([weights,] tsvector, tsquery [, normalization])	Ranking standard
ts_rank_cd([weights,] tsvector, tsquery [, normalization])	Ranking basato su copertura e posizione dei termini
ts_headline([config,] document, query [, options])	Estrae un frammento di testo con i termini evidenziati

```
SELECT
  titolo,
  ts_rank(
    ricerca,
    websearch_to_tsquery('italian', 'gatto veloce')
  ) AS rilevanza
FROM articoli
WHERE ricerca @@ websearch_to_tsquery('italian', 'gatto veloce')
ORDER BY rilevanza DESC;
```

```
SELECT
  ts_headline(
    'italian',
    corpo,
    websearch_to_tsquery('italian', 'gatto veloce'),
    'StartSel=<b>, StopSel=</b>, MaxFragments=2'
  ) AS anteprima
FROM articoli
WHERE ricerca @@ websearch_to_tsquery('italian', 'gatto veloce');
```

ts_rank_cd considera anche la vicinanza e la copertura dei termini nel documento; è spesso preferibile quando l’ordine e la densità delle parole contano.

4. Funzioni di supporto

Funzione	Uso
ts_lexize(dictionary, token)	Mostra come un dizionario normalizza un token
ts_stat(sqlquery [, weights])	Statistiche sui lexemi di una colonna tsvector
ts_delete(tsvector, lexeme)	Rimuove un lexeme dal vettore
ts_filter(tsvector, weights)	Seleziona solo i lexemi con determinati pesi
array_to_tsvector(text[])	Crea un tsvector da un array di stringhe
numnode(tsquery)	Conta i nodi dell'albero della query
querytree(tsquery)	Mostra la parte indicizzabile della query

```
-- Come viene normalizzata una parola
SELECT ts_lexize('italian_stem', 'cavalli');

-- Termini più frequenti e documenti che li contengono
SELECT *
FROM ts_stat('SELECT ricerca FROM articoli')
ORDER BY ndoc DESC
LIMIT 10;
```

ts_lexize è utile per diagnosticare stemming e stopword; ts_stat è utile per analizzare il vocabolario e costruire suggerimenti o filtri basati sui termini più frequenti.

5. Indicizzazione

Per rendere la ricerca veloce, crea una colonna tsvector generata e indicizzala con GIN. La colonna evita di ricalcolare parser e dizionari a ogni ricerca; l'indice GIN rende efficiente il match @@ anche su grandi volumi di testo.

```
ALTER TABLE articoli
ADD COLUMN ricerca tsvector
GENERATED ALWAYS AS (
  setweight(to_tsvector('italian', coalesce(titolo, '')), 'A') ||
  setweight(to_tsvector('italian', coalesce(corpo, '')), 'B')
) STORED;

CREATE INDEX idx_articoli_ricerca
ON articoli USING gin (ricerca);
```

Alternativa con trigger

```
CREATE TRIGGER trg_articoli_ricerca
BEFORE INSERT OR UPDATE OF titolo, corpo ON articoli
FOR EACH ROW EXECUTE FUNCTION
  tsvector_update_trigger(
    ricerca, 'pg_catalog.italian', titolo, corpo
  );
```

Scelta rapida

```
Input dell'utente
|
|-- Testo libero semplice ------------> plainto_tsquery()
|-- Frase esatta ---------------------> phraseto_tsquery()
|-- Sintassi stile Google ------------> websearch_to_tsquery()
|-- Query avanzata controllata ------> to_tsquery()
|
|-- Vuoi ordinare per rilevanza? ----> ts_rank() oppure ts_rank_cd()
|-- Vuoi evidenziare i termini? -----> ts_headline()
|-- Vuoi velocizzare le ricerche? ---> colonna tsvector + indice GIN
```

Pattern consigliato

Per una ricerca utente tipica, il pattern più robusto è: colonna tsvector generata, indice GIN, websearch_to_tsquery per l'input, ts_rank o ts_rank_cd per l'ordinamento e ts_headline per le anteprime dei risultati.